A SOURCE-PINNED, CHECKED PROPOSAL PACKET

Further Lean Proof Simplifications

`DiophantineConsistency`

Four additions beyond the existing proposal corpus

The strongest change replaces manual proofs of object-formula specifications with explicit compositional semantic normalization. A second patch shares one scale-parameterized Pell estimate between ordinary and native consumers. Two checked architectural options isolate integer recovery and internal certificate closure.

Prepared for	Vladimir Reshetnikov
Date	26 October 2026
Repository	VladimirReshetnikov/ProveIt
Source commit	`23f631ea07fff8632eb8cd92286f7621bbe051c0`
Toolchain	Lean 4.32.0, repository-pinned mathlib
Deliverables	Report, exact source overlays, patches, examples, evidence

Contents

1 Results and the counterfactual baseline **1**

2 What was excluded, and why these directions remain **1**

3 Normalize exact object-formula specifications compositionally **2**

3.1 The repeated proof obligation 2

3.2 A structural induction becomes a semantic fold 3

3.3 Measured scope and preservation 3

4 Factor the Pell comparison through a positive scale **4**

4.1 Keep the native denominator normalization visible 4

4.2 Why the generalization works 4

4.3 Reuse the lower comparison at the same boundary 5

4.4 Cost and relation to prior work 5

5 Optional: isolate integer recovery from two error budgets **5**

5.1 A shared discrete conclusion 5

5.2 Checked applications and limits 6

6 Optional: share closure of observed internal certificate sets **6**

6.1 The repeated operation is union and insertion 6

6.2 The two laws and their necessary hypotheses 6

6.3 What has actually been checked 7

7 Validation, reproduction, and adoption order **7**

7.1 Evidence and its scope 7

7.2 A concrete integration sequence 8

7.3 Commands and artifact map 8

A Exact specification targets **9**

1 Results and the counterfactual baseline

Recommendation. Adopt the local specification proofs first, then review the common Pell estimate and its exact ordinary/native adapters as one mathematical refactor. Keep integer recovery and observed-set closure as optional abstractions until their client population justifies the additional interface.

The two integrated patch families remove 223 physical source lines against the pinned checkout, counting new helpers and imports. The earlier corpus already proposed simplifying trivial existential aliases inside the Pell estimates. Charging that prior improvement to the baseline leaves 219 additional lines removed in the explicitly scoped counterfactual. This adjustment is not a claim to have merged and rebuilt every alternative design in the earlier packets.

Direction	Concrete scope	Validation level	Net lines
Compositional formula specifications	20 proofs; 8 modules	Complete production modules	−133
One scaled Pell ratio estimate	3 modules, shared theorem	Complete production modules	−86
Two-error integer recovery	Core and 3 applications	Checked separate example	Unclaimed
Observed certificate-set closure	Core and 6 client theorems	Checked separate examples	Unclaimed

Table 1: Line reductions include maintained helper costs. The Pell figure is the incremental reduction after the prior alias cleanup. Optional examples are not included in the total.

The requested Lean tree contains 618 files and 80,297 physical lines at the pin, including comments and blank lines. The deliverable changes 24 existing proofs in 11 production files; its common helper is added to an existing module. One of the existing files is the lower-level Hilbert-tenth-problem Pell ratio module: putting the common theorem there permits both existing consumers to use it without reversing the import direction. Two optional closure client families also live in imported foundational modules outside the requested tree; their possible savings are not counted.

The source snapshot is the 26 October commit `23f631ea07fff863`. The patch retains public theorem statements and the existing mathematical objects. It does not change a Diophantine equation, coefficient, witness count, tile table, encoded formula, or native factor of two. Source compression here is an engineering result, not a stronger consistency theorem or a measured compilation-speed improvement.

2 What was excluded, and why these directions remain

The audit treats *all* suggestions in the two named locations as already available: adopted changes, pending patches, uncompiled sketches, deferred architecture, and supplementary research directions. Reimplementing an old idea with a new theorem name does not qualify. The supplied `evidence/novelty-audit-initial.txt` records 53 initial excluded families. The three `evidence/late-audit-part*.txt` files extend that review to the four incoming archives added during this task. Inventories record the archive and source hashes.

The scope includes the six research packets 2026-10-26a through 2026-10-26f, the nine proof-simplification archives physically present in `docs/incoming`, and relevant index descriptions. The index also links historical mathematics archives elsewhere in the repository; this report does not claim a semantic review of every transitively linked historical archive. The incoming ZIPs were read as inert data; their supplied scripts were not executed.

The final refresh changed the recommendations. The source and all production candidates were rebased to `23f631ea07fff863`. The new `additional_lean_simplifications` archive already proposes a stronger fundamental-unit simplification for the exact client, so the small-square alternative was withdrawn. Round2's screening entry N08 and New Deltas' supplementary entry X3 already propose rail/rank reuse, so the finite-cut family was withdrawn too. Finally, the new-simplifications archive deletes `zeroSumStage_spec` and `zeroProductStage_spec` by eliminating redundant inductions; those two targets were removed from the specification patch. Tentative status or a different proof does not make a prior idea new.

New contribution	Nearest already-proposed work	Additional obligation addressed here
Formula-specification normalization	Definable-induction wrappers, model-validity bridges, internal recursion preservation, constructor views	Deriving the exact `Sat` specification under binders; the earlier wrappers consume it.
Scaled Pell estimate	Source-register decoding and trivial `set`/`let` alias cleanup	One analytic error estimate parameterized by a positive scale, with exact ordinary and doubled clients.
Integer recovery	Half-radix identities and cyclic carry normalization	A unit-interval uniqueness theorem with an explicit sum of two real error budgets.
Observed certificate closure	Guarded sequential history folds, internal constructor inversion and unique-existence induction	Union and singleton insertion for arbitrary internal sets, observed through their pair/triple encodings.

Several tempting changes were therefore rejected: generic sum-of-squares lemmas, degree automation, circuit certificates, decoder fusion, Pell parity reuse, functional-graph witness elimination as a broad direction, and another induction transport wrapper. The current packet uses ordinary validation and scoped simplification practices, but does not count those practices as new ideas.

3 Normalize exact object-formula specifications compositionally

3.1 The repeated proof obligation

Many formulas are deliberately explicit: quantifier order, de Bruijn slots, naturalness guards, and represented arithmetic atoms are part of their meaning. Their specification theorems then prove that interpretation by `Sat` agrees with a semantic predicate. A manual proof often introduces every binder, splits an equivalence into two directions, constructs the same existential witnesses, applies an atom's specification using `.mp` or `.mpr`, and follows each environment projection separately.

These steps are mostly the recursive equations of satisfaction plus facts already proved about the atoms. The useful boundary is explicit, local normalization: unfold the selected outer formula and the logical clauses of `Sat`; rewrite its atomic graphs through their exact specifications; reduce `scons`; finish with only the propositional rules the formula needs. This does not require a custom tactic or a global simp policy.

For example, the complete replacement body for `commutativeStage_spec` is:

```
by
  simp only [commutativeStage, Sat, addForm_spec H, scons]
```

The successor-left specification also has determined successor witnesses. Its checked body makes their elimination explicit:

```
by
  simp only [successorLeftStage, Sat, addForm_spec H,
    fSuccF_spec H, scons, exists_eq_left]
```

This is not a suggestion to indiscriminately unfold every object-language definition. Each proof supplies its own finite list of semantic facts. In `boundAtomForm_spec`, naturalness and tag guards occur beside two determined successor witnesses. The proof additionally uses `exists_and_left` to expose those equalities and `exists_eq_left` to substitute them. The guards remain in the resulting proposition.

3.2 A structural induction becomes a semantic fold

The more substantial example is `evalForm_spec`. Its induction is necessary because the object formula is compiled recursively from a natural expression. What can disappear is the repeated binder and environment bookkeeping inside the addition and multiplication cases:

```
by
  induction e generalizing coordinates z ee with
  | constant n => exact fNumF_spec H n z ee
  | var i => rfl
  | add p q hp hq | mul p q hp hq =>
    simp only [evalForm, Sat, EvalRel, hp, hq,
      addForm_spec H, mulForm_spec H, Function.comp_def, scons]
```

Generalizing coordinates, the output slot, and the environment is retained: recursive formulas run beneath additional binders. Removing that generalization would make the induction hypotheses too weak. The proof still establishes the semantics of the actual formula compiler, even if a prior semiring abstraction has already organized its surrounding algebra.

3.3 Measured scope and preservation

Module	Proofs	Lines removed
`ObjectAdditionLaws`	3	20
`ObjectAdditionCancellation`	1	4
`ObjectMultiplicationLaws`	2	10
`ObjectDistributivity`	2	14
`ObjectNaturalOrder`	2	10
`ObjectNaturalExpression`	1	17
`ObjectCodeBound`	6	41
`ObjectCodeSeal`	3	17
Total	20	133

The eight modules shrink from 1,713 to 1,580 physical lines, removing 5,283 UTF-8 bytes. The selected proof bodies shrink from 185 to 52 lines. No helper, macro, or global simp declaration is

added. A deterministic erasure audit found every byte outside the selected proof bodies identical, including theorem statements, formula data, quantifier order, and executable syntax. All eight complete replacement modules compile together.

The model carrier remains arbitrary. No object-level natural is assumed standard, and no internal induction is replaced with external induction on Lean's `Nat`. Earlier preservation-stage targets in `ObjectArithmetic`, `ObjectMultiplication`, and `ObjectCodeClosure` are deliberately omitted. The separate totality and uniqueness specifications in `ObjectCodeBound` are also omitted because the earlier unique-existence proposal already targets that layer. The late induction-elimination proposal also removes the zero-sum and zero-product stage specifications; those targets are excluded rather than counted as remaining opportunities.

The principal maintenance risk is a future change in a formula's semantic normal form. An explicit simp list makes the dependency visible, and the unchanged theorem statement catches any resulting mismatch. The patch is small enough that introducing a new object-specification tactic would add more machinery than it removes.

4 Factor the Pell comparison through a positive scale

4.1 Keep the native denominator normalization visible

The ordinary ratio proof in `Jones1980.Ratio90` and the native proof in `upper_estimate_scaled` use the same Pell growth argument. The native source has a first coordinate twice the Pell sequence value. That factor must remain in the statement and in the final ratio.

Write, with $U, Y, r \geq 1$,

$$J = 2r + 1, \qquad a = Y(U+1), \qquad A = a + 2, \qquad P = 2UY^2 + 1,$$

$$c = \psi_A(J), \qquad k_0 = \psi_P(r+1), \qquad \xi = \frac{(U+1)^{2r}}{U^r}.$$

The common theorem, named `upper_estimate_budget`, accepts a positive real scale λ (spelled `scale` in Lean). Its distinctive hypothesis and conclusion are

$$\xi < \lambda(Y+1) \quad \Longrightarrow \quad \frac{c}{k_0} < \xi + \frac{\lambda}{2},$$

under the existing growth assumptions $72r < U + 1$ and $16 \leq Y(U+1)$, and the same Pell identities and positivity premises as the old proof.

4.2 Why the generalization works

Set $\varepsilon = 9/[Y(U+1)]$. The shared analytic work establishes

$$\frac{c}{k_0} \leq \xi(1 + 2r\varepsilon), \qquad (Y+1)\,2r\varepsilon < \frac{1}{2}.$$

The first statement follows from the existing upper/lower growth estimates, the polynomial comparison `upper_key`, and `one_add_pow_le`. The second follows from the same numerical guard $72r < U + 1$. Since $\lambda > 0$ and $2r\varepsilon \geq 0$,

$$\xi\,2r\varepsilon \leq \lambda(Y+1)\,2r\varepsilon < \frac{\lambda}{2}.$$

The final upper bound is then a single algebraic conclusion. The long power-and-denominator argument is proved once; the normalization scale enters only in the last error estimate.

Specializing to $\lambda = 1$ gives the old ordinary `upper_estimate'` statement. Specializing to $\lambda = 2$ gives the old native `upper_estimate_scaled` statement. The existing doubled-coordinate consumer then uses $k = 2k_0$ to obtain

$$\frac{c}{k} < \frac{\xi}{2} + \frac{1}{2}.$$

All original public arguments remain accepted, including an ordinary-wrapper premise that is not needed by the new proof. This avoids a gratuitous downstream signature migration.

4.3 Reuse the lower comparison at the same boundary

The native `lower_key` has the exact statement of the lower-level `Ratio90.lower_key`; its local polynomial proof can be an application. More significantly, `doubled_ratio_lower` follows from `Ratio90.lower_estimate` applied at k_0, multiplying by two and rewriting the native equation $k = 2k_0$. Its current public statement is preserved.

The new import goes from `NativePellPower` to the existing H10 ratio module. That module has no return dependency on `DiophantineConsistency`. The full overlay check rebuilds the intervening native scale modules before checking `NativeHalfBinomial`, so validation does not accidentally use an old compiled native theorem beneath a changed dependency.

4.4 Cost and relation to prior work

The three affected complete modules shrink by 90 lines against the pin, counting the helper once and the new import. Earlier reports already suggested replacing existential aliases for ξ, X, Z, ε with `set` or `let`. After charging the valid cleanup, including its redundant rewrite removal, to the baseline, the additional saving is 86 lines. Current source already includes the two-line cleanup inside the native `lower_key`; that reduction is already in the baseline and is not deducted twice. The source and counterfactual metrics are supplied separately so that this small overlap is not hidden.

The contribution is the scale-parametric inequality and its two mathematical specializations, together with exact lower-comparison reuse. It does not replace the earlier Pell-register decoding abstraction. The main review question is whether this common theorem belongs in the shared H10 module; the dependency direction and both existing clients support that placement. No sharper constants or broader optimality claim is made.

5 Optional: isolate integer recovery from two error budgets

5.1 A shared discrete conclusion

The end of `half_binomial_of_ratio` combines a binomial decomposition, an evenness argument, and two real inequalities to identify a natural number. The ordinary `Ratio90.Y_eq_floor` has the same final discrete task with different error constants.

The checked example separates that task. Let $Y, t \in \mathbb{N}$ and $\rho, \delta, \epsilon \in \mathbb{R}$, and assume

$$\delta \geq 0, \qquad \delta + \epsilon \leq 1,$$

$$Y < \rho < Y + 1, \qquad t + \delta < \rho < t + \delta + \epsilon.$$

Then $Y = t$. The second certificate implies $t \leq \rho < t + 1$, so `Nat.floor` of ρ is both Y and t. The proof uses `Nat.floor_eq_iff` twice and transports equality; it does not reprove properties of floors.

This interface exposes which analytic facts actually matter to integer recovery. A separate assumption $\epsilon \geq 0$ is unnecessary: the strict interval premises already imply a positive width. The proof does need nonnegativity of δ, and its total-error bound must be charged to the same unit interval.

5.2 Checked applications and limits

`examples/IntervalExtraction.lean` contains the core and three checked applications:

1. Native half-binomial recovery takes $\delta = f/2$ and $\epsilon = 1/2$ with $0 \leq f < 1$. The zero endpoint is allowed, matching the actual decomposition theorem.
2. Ordinary binomial recovery takes $0 \leq \delta < 1/4$ and $\epsilon = 1/2$.
3. A variable-denominator extension takes $s \geq 2$, $\delta = f/s$, and $\epsilon = 1/s$, again with $0 \leq f < 1$.

The budget is substantive. If it is allowed to exceed one, $t = 0$, $Y = 1$, $\rho = 5/4$, $\delta = 0$, and $\epsilon = 3/2$ satisfy both strict intervals but give unequal integers. If negative δ is allowed, $t = 1$, $Y = 0$, $\rho = 1/2$, $\delta = -3/4$, and $\epsilon = 1/2$ give another counterexample. These examples explain the contract; they are not new conditions added to the existing clients, which already meet it.

The actual parity proof, divisibility by two, and real decomposition remain the clients' responsibility. The supplied file checks the final client-shaped implications; it does not replace the complete production modules with a new rounding API. For only two consumers, the common lemma and adapters may add lines. Its value is a stable mathematical boundary for further normalization variants, so no net saving enters the total.

6 Optional: share closure of observed internal certificate sets

6.1 The repeated operation is union and insertion

`boundClosed_cup` and `boundClosed_insert` establish closure when two internal certificate sets are united or one justified observation is adjoined. The imported `InternalSoundness` renaming certificates and `CodedRank` rank certificates repeat that pattern. Their local step predicates differ, but each is monotone as the supporting internal set grows.

Use an observation type I, the arbitrary model carrier V, and its membership relation. The interface stores an encoder $e : I \to V$, a step predicate $\mathrm{step}(S, i)$, and monotonicity under internal set inclusion. Define

$$\mathrm{Closed}(S) \iff \forall i,\ e(i) \in S \implies \mathrm{step}(S, i).$$

For bound certificates, $I = V \times V$, $e(c, b)$ is the existing ordered-pair encoding, and the step predicate includes both $b \in \omega$ and the existing `BoundStep`. No guard is discarded to obtain a cleaner abstraction.

6.2 The two laws and their necessary hypotheses

If S and T are closed, an observed member of $S \cup T$ lies in one of the inputs. The corresponding step follows from that input's closure and then transports along its inclusion into the union. This proves union closure once for every rule family.

For insertion, assume e injective, S closed, and $\mathrm{step}(S, i)$. Consider an observation in $S \cup \{e(i)\}$. An old member is handled by monotonicity. A member of the new singleton has encoding equal to

$e(i)$; injectivity identifies its observation with i, and the supplied step transports to the enlarged set.

For unrestricted step predicates, injectivity is essential to this insertion interface. If two different observations have the same encoding, adjoining that encoding can create an obligation for the other observation, whose step was never supplied. The concrete adapters prove injectivity using the repository's `kpair_inj` or `satTriple_inj`; the generic theorem does not silently assume it.

The encoding also need not cover the carrier. Unrelated or malformed members of an internal set are allowed, just as in the original closure definitions: only encoded observations trigger obligations. Requiring every member to be a well-formed pair would strengthen the certificate and change the public meaning. Replacing the model's internal set with an external `Finset` would likewise be unjustified.

6.3 What has actually been checked

`examples/ObservedClosure.lean` supplies the kernel, including the empty set, union, and insertion. `examples/ClosureClients.lean` supplies six concrete client theorems: union and insertion for bound, renaming, and rank certificates. Both complete example files compile against the pinned repository's actual foundational definitions.

Only the bound family is in the requested folder. The renaming and rank families are explicit possible amortization sites in imported foundational code; they are not silently counted as reductions in `DiophantineConsistency`. The production patch does not introduce this architecture, and no net source reduction is claimed. Adoption becomes attractive when several families use the kernel and stop maintaining their own insertion arguments.

The prior sequential scanner-fold abstraction has a changing accepted-history state and list execution. The present laws concern unordered internal sets and their union/singleton operations. Closure alone does not establish well-foundedness, termination, or correctness of certificates; the existing internal induction and constructor-inversion arguments keep those roles.

7 Validation, reproduction, and adoption order

7.1 Evidence and its scope

The compiler is the repository's Lean 4.32.0. The mathlib commit is `81a5d257c8e410db22 7a6665ed08f64fea08e997`; the validation scripts check every dependency revision against `lake-manifest.json`. Repository prerequisites were built from the pinned sources in an isolated output tree. The final common overlay then checked all changed production modules and their affected local dependencies together.

The final overlay's import closure contains 71 repository modules. Its record distinguishes newly compiled outputs from unchanged baseline outputs reused only after source, dependency-object, and output hashes matched. This is an import-closure check, not a rebuild of every target in the repository. The package dependencies use their already-built artifacts at the verified revisions.

For the specification family, 16 additional full-source audit compilations printed the axioms of the 20 changed theorems before and after. Each retained exactly `propext`, `Classical.choice`, and `Quot.sound`. The other new examples and the final integrated endpoints were also checked with `##print axioms`; their reported dependencies contain no `sorryAx` or newly postulated project axiom. Ordinary Lean foundational axioms are not described as an axiom-free proof.

The source metrics count physical lines, including imports, comments, and blank lines. They are

reproducible for the shipped candidates. The counterfactual Pell adjustment is narrower: it applies the known prior alias cleanup before calculating the additional reduction. Earlier architectural packets contain alternatives and prospective designs, so this report does not invent a globally combined post-proposal checkout. The novelty claim is about the new mechanisms and obligations, with that limitation made explicit.

Elapsed times in raw logs are diagnostic data only. Shared caches, concurrent checks, and local resource conditions make them unsuitable for a performance comparison. An interrupted baseline build during disk-space recovery was rerun from regenerated outputs; the final successful records identify the artifacts used in the overlay.

7.2 A concrete integration sequence

1. Apply the formula-specification patch. It changes proof bodies only, has no helper cost, and gives the largest immediate reduction.
2. Review the common Pell theorem together with all four public adapters. Keep the factor of two explicit and rebuild native scale consumers.
3. Revisit the two optional abstractions when there are enough clients to amortize their contracts, encoders, and adapters. Do not count two-line applications while ignoring the new abstraction's source.

The patches target the pinned checkout. If earlier proposals have already changed the same lines in a working branch, port the mechanism into their resulting interfaces and rerun the supplied checks; do not overwrite those changes with the full candidate snapshots. The snapshots exist to make this review exact and independently reproducible.

7.3 Commands and artifact map

The included `scripts/validate.py` constructs an isolated source overlay without changing the repository. It checks the repository pin, Lean version, and dependency commits; discovers local imports; and compiles the selected closure. Dependency package artifacts must already be available, as for a normal local Lean project build. A typical invocation is:

```
python3 scripts/validate.py \
  --repo /path/to/ProveIt \
  --lean /path/to/lean-4.32.0/bin/lean \
  --packages /path/to/ProveIt/.lake/packages \
  --out /tmp/diophantine-simplification-check

python3 scripts/check_examples.py \
  --repo /path/to/ProveIt \
  --lean /path/to/lean-4.32.0/bin/lean \
  --packages /path/to/ProveIt/.lake/packages \
  --validated /tmp/diophantine-simplification-check
```

The README gives the exact prerequisites and explains the optional verified baseline reuse. The shipped evidence contains the actual successful run, rather than claiming these portable commands built a full fresh dependency cache. PDF reproduction uses two `pdflatex` passes on `report.tex`.

Artifact	Purpose
`report.tex`, `report.pdf`	Editable manuscript and rendered report, with exact source links.
`metrics.tex`	Generated numerical values used by the manuscript.
`patches/combined.patch`	One reviewable patch for the two integrated directions.
`patches/`	Independent family patches for selective adoption.
`candidates/`	Complete proposed production sources, preserving repository-relative paths.
`examples/`	Checked optional cores and small independent demonstrations.
`evidence/`	Novelty audit, inventories, source metrics, compiler records, and axiom reports.
`scripts/`	Pin-aware compilation, example/axiom checks, and integrity/metric verification.
`validation-plan.json`	Exact production endpoints and example/audit targets.
`SHA256SUMS`	Integrity hashes for all other files in the packet.

A Exact specification targets

The proof-only patch changes the following names in their existing module namespaces. Original source ranges and before/after hashes are recorded in the machine-readable theorem and module metrics.

Module	Target declarations
`ObjectAdditionLaws`	`successorLeftStage_spec`, `commutativeStage_spec`, `associativeStage_spec`
`ObjectAdditionCancellation`	`cancellationStage_spec`
`ObjectMultiplicationLaws`	`successorLeftStage_spec`, `commutativeStage_spec`
`ObjectDistributivity`	`distributiveStage_spec`, `associativeStage_spec`
`ObjectNaturalOrder`	`additionBoundStage_spec`, `differenceStage_spec`
`ObjectNaturalExpression`	`evalForm_spec`
`ObjectCodeBound`	`boundAtomForm_spec`, `boundBinaryForm_spec`, `boundQuantifierForm_spec`, `boundStepForm_spec`, `boundClosedForm_spec`, `boundCodeForm_spec`
`ObjectCodeSeal`	`sealCodeForm_spec`, `sealedZFCAxiomCodeForm_spec`, `ctxSealedZFCAxiomsForm_spec`